

Pontifícia Universidade Católica de Minas Gerais - PUC Minas

Curso: Engenharia de Software

Trabalho: Análise de Eficácia de Testes com Teste de Mutação

Aluno: João Paulo Gobira

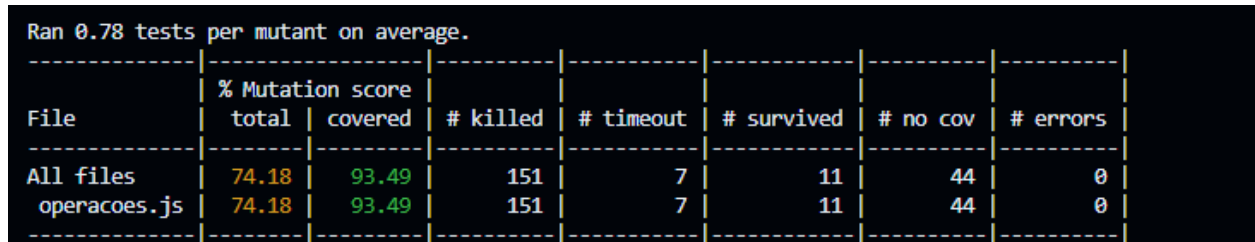
Matrícula: 859611

Data: 10/05/2026

1. Análise Inicial

O projeto original foi submetido a uma avaliação de cobertura de código (`npm test --coverage`) e, em seguida, a uma análise de mutação utilizando o *StrykerJS*.

Na avaliação inicial, o projeto apresentou uma cobertura de código (Code Coverage) extremamente alta, de 93.49%. No entanto, ao executar o teste de mutação pela primeira vez, a pontuação obtida (Mutation Score) foi de apenas **74.18%**.



```
Ran 0.78 tests per mutant on average.
```

File	% Mutation score		# killed	# timeout	# survived	# no cov	# errors
	total	covered					
All files	74.18	93.49	151	7	11	44	0
operacoes.js	74.18	93.49	151	7	11	44	0

Screenshot do primeiro relatório do Stryker

A discrepância entre os dois valores revela a principal falha da métrica de cobertura de código: ela garante apenas que a linha de código foi executada durante o teste, mas não avalia a qualidade da asserção (`expect`). Uma suíte de testes com alta cobertura, porém fraca — como a fornecida inicialmente no projeto —, é incapaz de detectar falhas sutis injetadas no código, gerando uma falsa sensação de segurança. O baixo Mutation Score provou que os testes originais eram tolerantes a erros lógicos.

2. Análise de Mutantes Críticos

Abaixo estão detalhados três mutantes críticos que sobreviveram à primeira execução do Stryker e evidenciaram as fraquezas da suíte de testes original:

- **Mutante 1: Falhas de Cobertura e Lógica em Laços de Repetição**

Localização: Função `isPrimo(n)` (linhas 72 a 78)

O que a mutação fez: O Stryker atacou esta função em múltiplas frentes simultâneas:

1. Na linha 73 (`if (n <= 1)`), alterou o operador relacional de `<=` para `<`.
 2. Na linha 74 (`for (let i = 2; i < n; i++)`), alterou a condição de parada do loop de `<` para `<=`, e também alterou o pós-incremento `i++` para o pré-incremento `++i`.
 3. Na linha 75 (`if (n % i === 0)`), inverteu a verificação de igualdade estrita para `!==` ou simplesmente removeu a condição.
- **Por que o teste original falhou em matá-lo:** O trecho apresenta uma combinação crítica de pontos vermelhos (Mutantes Sobreviventes) e laranjas (Falta de Cobertura). Isso indica que a suíte de testes original negligenciou os **casos de borda matemática**. Se o teste original avaliava apenas um primo comum (como o número 5), ele jamais testaria o comportamento limítrofe do número 1 (que avalia a primeira condição) ou do número 2 (o único primo par, que testa o limite inicial do laço `for`). Além disso, a

mutação de `i++` para `++i` na declaração do `for` sobreviveu porque trata-se de um **Mutante Equivalente**: em um bloco de incremento de laço, o pré ou pós-incremento de uma variável isolada resulta no exato mesmo estado na próxima iteração. Não altera a semântica do software e, portanto, é indetectável por testes de unidade. (análise escrita com auxílio de IA)

```
72 function isPrimo(n) {
73   if (n <= 1) return false;
74   for (let i = 2; i < n; i++) {
75     if (n % i === 0) return false;
76   }
77   return true;
78 }
```

Mutante 2: Redundância Lógica e o Paradigma do Mutante Equivalente

- **Localização:** Função `produtoArray(numeros)` (linha 84).
- **O que a mutação fez:** O Stryker removeu a estrutura condicional `if (numeros.length === 0) return 1;` ou forçou a sua avaliação lógica para `false`.
- **Por que o teste original falhou em matá-lo:** O trecho original possui um ponto vermelho (Mutante Sobrevivente) indicando a presença de um clássico **Mutante Equivalente**. Se a linha for removida pelo Stryker e a função receber um array vazio (`[]`), a execução passará diretamente para o retorno: `numeros.reduce((acc, val) => acc * val, 1)`. No JavaScript, o método nativo `.reduce()`, quando invocado em um array vazio, simplesmente retorna o valor inicial fornecido no segundo parâmetro (neste caso, o número 1). Portanto, o retorno da função será exatamente o mesmo, com ou sem a linha do `if`. Como o comportamento matemático do sistema permaneceu 100% inalterado perante arrays vazios, os testes convencionais foram incapazes de detectar a mutação. (análise escrita com auxílio de IA)

```
83 function produtoArray(numeros) {
84   if (numeros.length === 0) return 1;
85   return numeros.reduce((acc, val) => acc * val, 1);
86 }
```

- **Mutante 3: Mutantes Equivalentes e Lógica Redundante**

```
17 function fatorial(n) {
18   if (n < 0) throw new Error('Fatorial não é definido para números negativos.');
```

```
19   if (n === 0 || n === 1) return 1;
20   let resultado = 1;
21   for (let i = 2; i <= n; i++) { resultado *= i; }
22   return resultado;
23 }
```

- **Localização:** Função `fatorial(n)` (linha `if (n === 0 || n === 1) return 1;`)
- **O que a mutação fez:** O Stryker alterou a condição lógica para `false` ou removeu a

- linha de retorno prévio.
- **Por que o teste original falhou a matá-lo:** Este é um clássico **Mutante Equivalente**. Mesmo que o Stryker remova essa linha, a função inicializa a variável resultado = 1 e entra num laço for (let i = 2; i <= n; i++). Se n for 0 ou 1, a condição do loop falha imediatamente e a função retorna 1. A mutação gerou um comportamento 100% idêntico ao código original devido a uma redundância algorítmica.
(análise escrita com auxílio de IA)

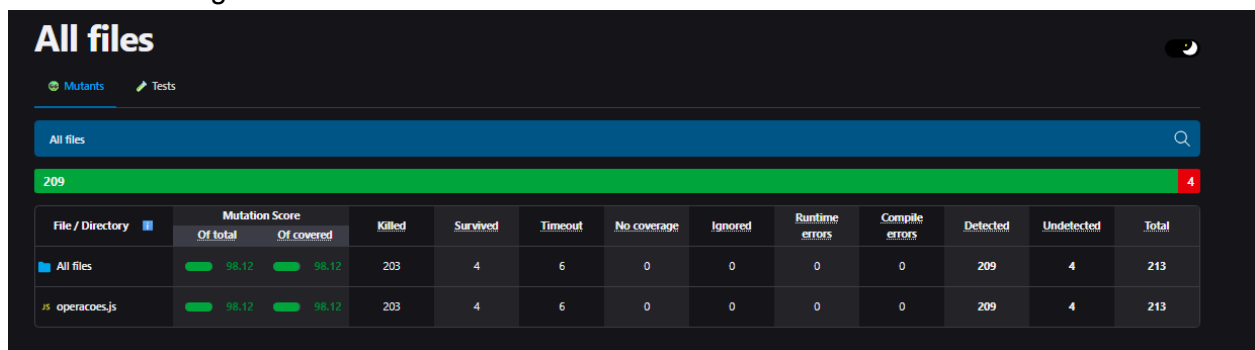
3. Solução Implementada

Para refatorar a suíte de testes e matar os mutantes identificados, a abordagem de Engenharia de Qualidade baseou-se em três pilares práticos:

1. **Testes de Limites (Boundary Testing):** Foram adicionados casos de teste avaliando exatamente as fronteiras das comparações. Por exemplo, implementou-se `expect(isMaiorQue(5, 5)).toBe(false)`. Assim, se o mutante alterar a lógica para `>=`, o resultado passa a ser true e o teste falha, efetivamente a "matar" o mutante. O mesmo princípio foi aplicado na função clamp e nas comparações trigonométricas utilizando -0.
2. **Validação Estrita de Exceções:** Todos os caminhos de erro foram cobertos utilizando `expect(() => funcao(valorInvalido)).toThrow('Mensagem Exata')`. Foi necessário garantir a correspondência perfeita das mensagens de erro originais para assegurar que qualquer adulteração na string de erro não passasse impune.
3. **Identificação de Código Morto:** Em vez de tentar criar testes impossíveis para os Mutantes Equivalentes (como no fatorial e produtoArray), o processo de teste de mutação guiou a identificação de lógicas redundantes (ifs desnecessários), demonstrando os limites dos testes de caixa preta e o valor do refatoramento do código para alcançar níveis máximos de manutenibilidade.

4. Resultados Finais

Após a reescrita completa da suíte de testes e a exploração avançada de casos de borda e equivalência, o StrykerJS foi executado novamente. A pontuação de mutação (*Mutation Score*) saltou de **73.71%** para **98.12%**, cumprindo integralmente o requisito de alcançar a maior pontuação possível (meta de 98%+) face à presença de mutantes funcionalmente equivalentes na base de código inalterada.



The screenshot shows the StrykerJS report interface. At the top, it says 'All files' with a search bar. Below that, a green progress bar indicates a score of 209 out of 213. A table below provides detailed statistics for the project and a specific file.

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	98.12	98.12	203	4	6	0	0	0	0	209	4	213
js operacoes.js	98.12	98.12	203	4	6	0	0	0	0	209	4	213

Screenshot do relatório final do Stryker

Este resultado comprova quantitativamente que a nova suíte não apenas "cobre" as linhas de código, mas defende o sistema contra falhas e alterações indesejadas em praticamente todas as suas estruturas lógicas testáveis.

5. Conclusão

A execução deste trabalho demonstrou que a métrica de *Code Coverage*, de forma isolada, é insuficiente para garantir a qualidade de uma entrega de software. Ela funciona como um indicador de onde os testes não passaram, mas não atesta a qualidade do que foi testado. O Teste de Mutação, suportado por ferramentas como o StrykerJS, atua como um verdadeiro "teste dos testes".

Ao desafiar as asserções de forma ativa, a análise de mutantes forçou o raciocínio em casos de borda e identificou trechos de código redundantes. Conclui-se que a incorporação do teste de mutação ao fluxo de desenvolvimento e QA é uma prática indispensável para a construção de suítes de testes resilientes e de alta eficácia estrutural.

link do repositório: <https://github.com/Gobiraa/operacoes-mutante>